Run `makemigrations` to create 0010_promoter_address.py. Then, run it again, using the `--empty promoters` option to create a new migration file for your data operations. I renamed mine to 0011_data_address.py, which is shown in the following listing, including the added data operation.

Listing E.46 A migration script with a data operation in it

```
# RiffMates/promoters/migrations/0011_data_address.py
from django.db import migrations

def group_address(apps, schema_editor):
    Promoter = apps.get_model("promoters", "Promoter")
    for promoter in Promoter.objects.all():
        text = ""
        if promoter.street_address:
            text += promoter.street_address + "\n"
        if promoter.city:
            text += promoter.city + "\n"
        if promoter.province_state:
            text += promoter.province_state + "\n"
        if promoter.country_code:
            text += promoter.country_code + "\n"
        if promoter.postal_zip:
            text += promoter.postal_zip + "\n"

        promoter.address = text
        promoter.save()

class Migration(migrations.Migration):
    dependencies = [
        (
            "promoters",
            "0010_promoter_address",
        ),
    ]

    operations = [
        # Can't undo this migration, there is no way to know which
        # lines were missing in the grouping
        migrations.RunPython(group_address),
    ]
```

This function is called by migrations.RunPython to perform data operations.

Iterate over all promoters.

The apps.get_model() function gets the version of the Promoter model that is correct at this step in the migration process.

Create a string based on the combined values of the address fields.

Copy the string into the new field, and save the model.

Dependencies are created automatically from the makemigrations command.

As there is no way to undo this data operation, the RunPython command only takes a single reference: the group_address function.

Once your data migration is complete, you can modify `Promoter` to remove the now extra address fields. Running `makemigrations` on that will result in the new file: 0012_remove_promoter_city_remove_promoter_country_code_and_more.py.

E.10 Chapter 11

Adding an API to your project enables you to provide access to your data to other systems, whether those are consumers of your data or alternate user interfaces, such as mobile apps or single-page applications (SPAs).